

머신러닝 프로그래밍

Machine Learning Programming

Lecture 10-2 :Random Forest Model



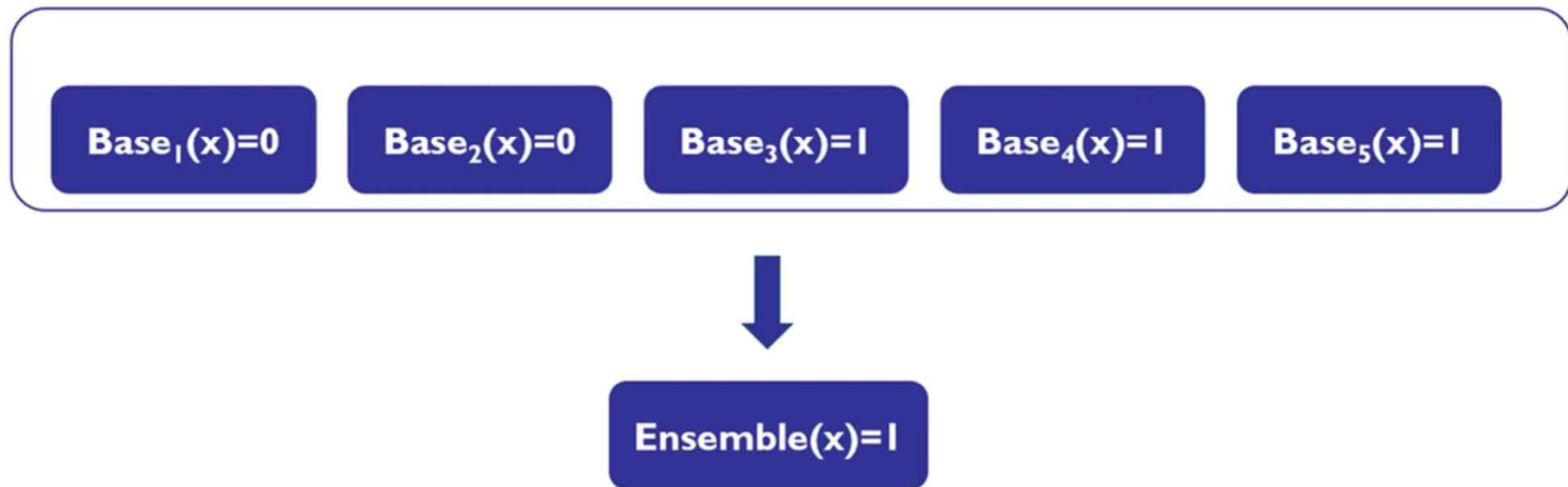
충북대학교
CHUNGBUK NATIONAL UNIVERSITY



충북대학교
산업인공지능연구센터

랜덤 포레스트 배경- 앙상블

- 여러 Base 모델들의 예측을 다수결 법칙 또는 평균을 이용해 통합하여 예측 정확성을 향상시키는 방법



- 다음 조건을 만족할 때 앙상블 모델은 Base 모델보다 우수한 성능을 보여줌
 - Base 모델들이 서로 독립적
 - Base 모델들이 무작위 예측을 수행하는 모델보다 성능이 좋은 경우

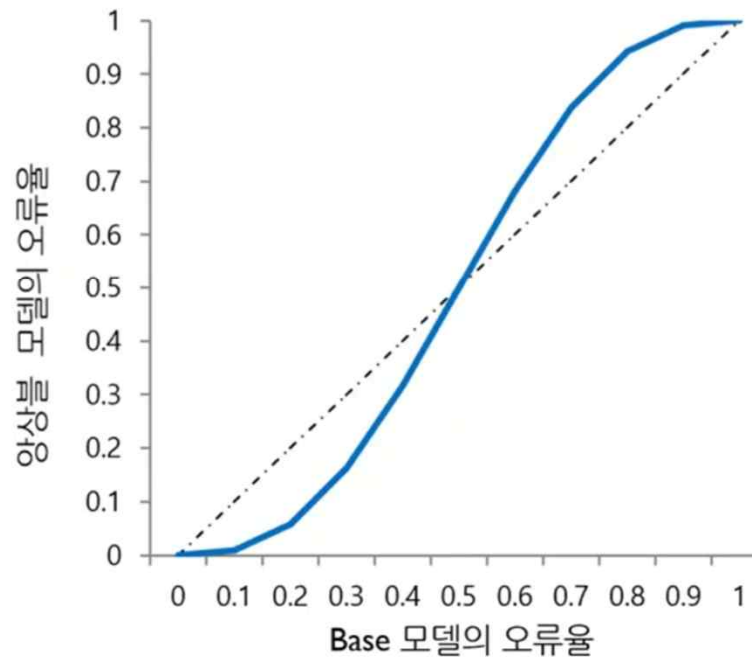
랜덤 포레스트 배경- 앙상블

- 앙상블 모델의 오류율

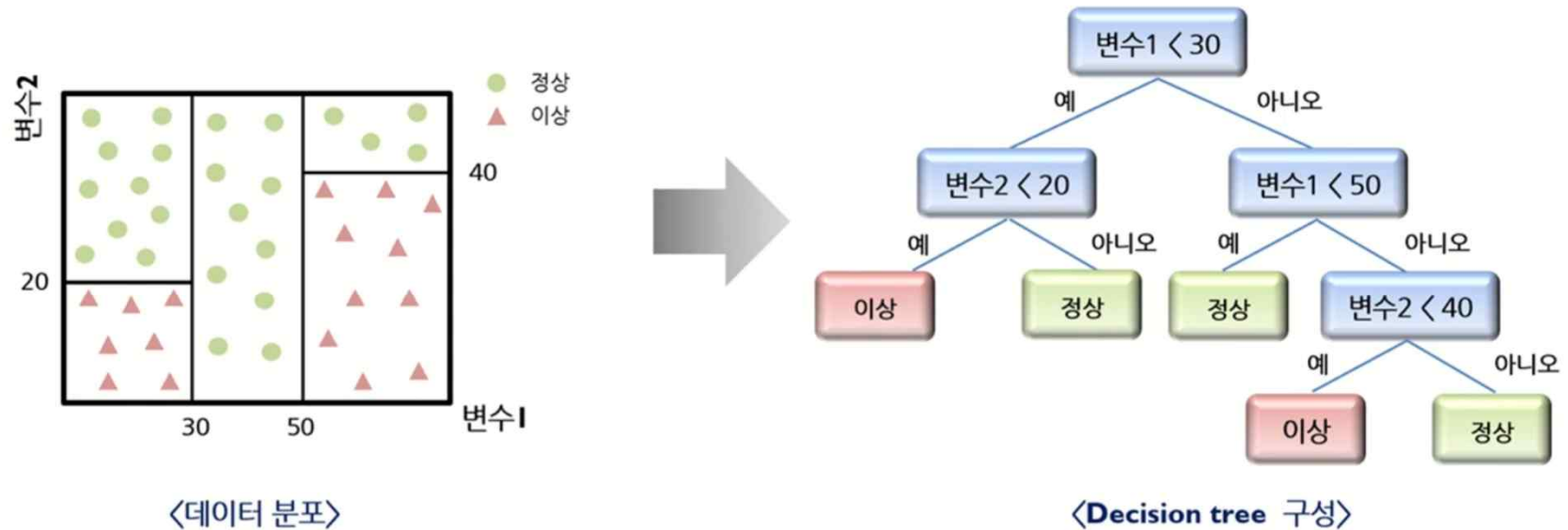
$$e_{ensemble} = \sum_{i=\lceil N/2 \rceil}^N \binom{N}{i} e^i (1-e)^{N-i}$$

e : Base 모델의 오류율
 N : Base 모델의 수

- 예를 들어, 5개의 binary classifier를 base 모델로 가지는 앙상블 모델의 오류율은 아래 그림과 같이 나타남. Base모델의 성능이 무작위 모델 보다는 좋아야 함.



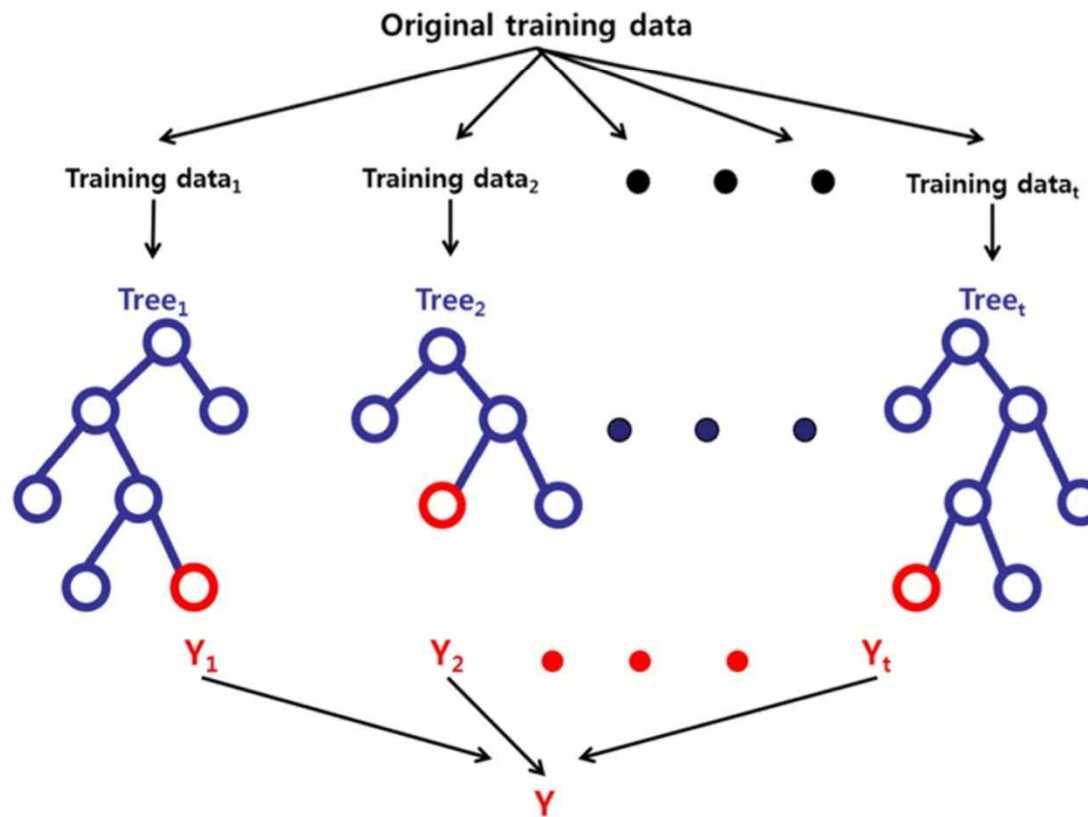
랜덤 포레스트 배경- 앙상블



- 의사결정나무모델은 앙상블 모델의 base 모델로서 활용도가 높음 (Maclin et al., 1999)
 - Low computational complexity: 데이터의 크기가 방대한 경우에도 모델을 빨리 구축할 수 있음
 - Nonparametric: 데이터 분포에 대한 전제가 필요하지 않음

랜덤 포레스트 개요

- 다수의 의사결정나무모델에 의한 예측을 종합하는 앙상블 방법
- 일반적으로 하나의 의사결정나무모델 보다 높은 예측 정확성을 보여줌
- 관측치 수에 비해 변수의 수가 많은 고차원 데이터에서 중요 변수 선택 기법으로 널리 활용됨



1. **Bootstrap** 기법을 이용하여 다수의 **training data** 생성

2. 생성된 **training data**로 **decision tree** 모델 구축 (무작위 변수를 사용하여)

3. 예측 종합

랜덤 포레스트 개요

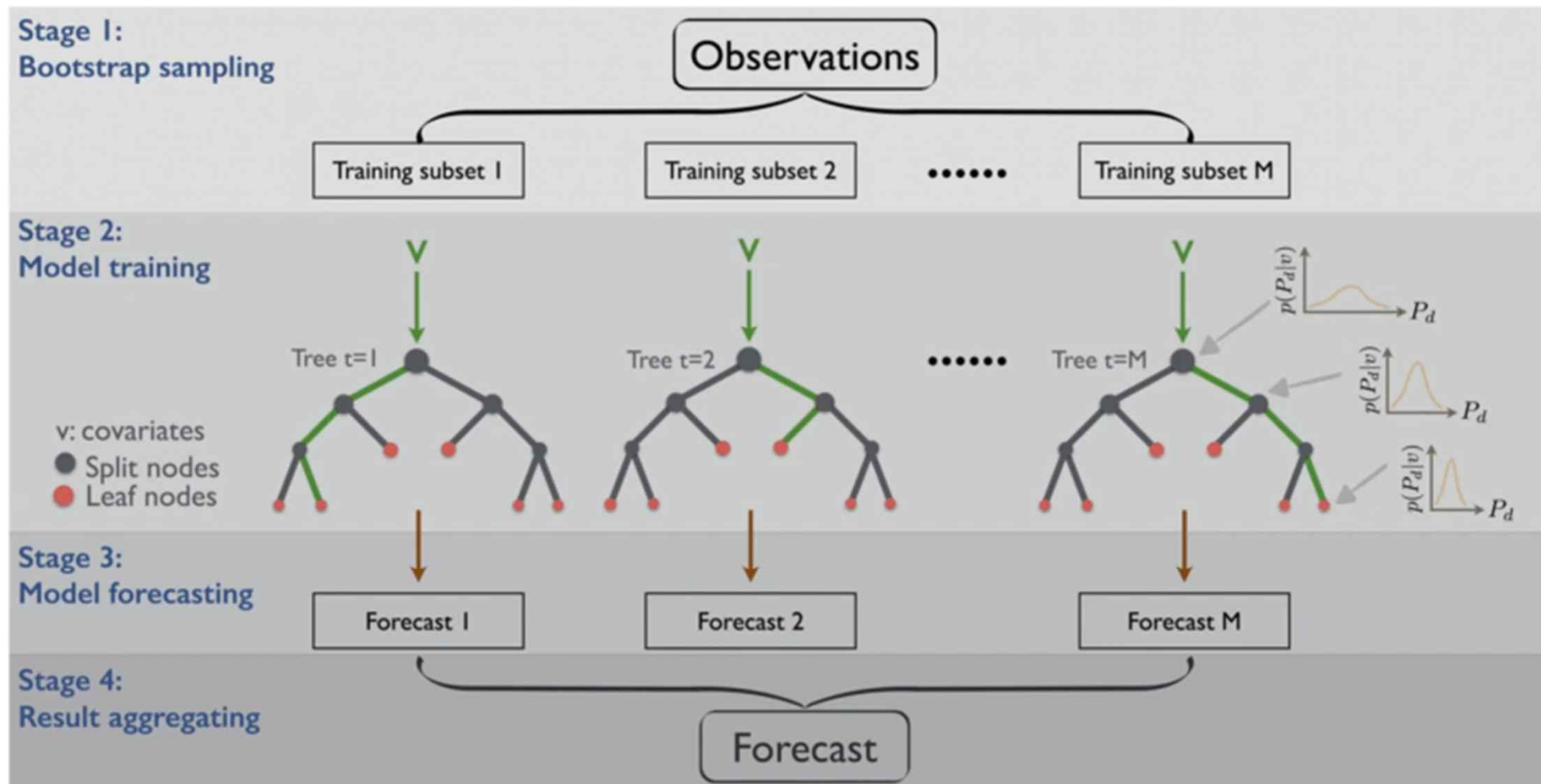
- 핵심 아이디어: Diversity, Random 확보

1. 여러 개의 Training data를 생성하여 각 데이터마다 개별 의사결정나무모델 구축 → **Bagging**

2. 의사결정나무모델 구축 시 변수 무작위로 선택
→ **Random subspace**

Bagging (Bootstrap Aggregating)

- Bagging (**B**ootstrap **A**ggregating): 각각의 bootstrap 샘플로부터 생성된 모델을 합침



Bagging (Bootstrap Aggregating)

■ Bootstrapping

- 각 모델은 서로 다른 학습 데이터셋을 이용
- 각 데이터셋은 복원 추출(sampling with replacement)을 통해 원래 데이터의 수만큼의 크기를 갖도록 샘플링
- 개별 데이터셋을 붓스트랩셋이라 부름

Original Dataset

x^1	y^1
x^2	y^2
x^3	y^3
x^4	y^4
x^5	y^5
x^6	y^6
x^7	y^7
x^8	y^8
x^9	y^9
x^{10}	y^{10}

Bootstrap 1

x^3	y^3
x^6	y^6
x^2	y^2
x^{10}	y^{10}
x^8	y^8
x^7	y^7
x^7	y^7
x^3	y^3
x^2	y^2
x^7	y^7

Bootstrap 2

x^7	y^7
x^1	y^1
x^{10}	y^{10}
x^1	y^1
x^8	y^8
x^6	y^6
x^2	y^2
x^6	y^6
x^4	y^4
x^9	y^9

...

Bootstrap B

x^9	y^9
x^5	y^5
x^2	y^2
x^4	y^4
x^7	y^7
x^2	y^2
x^5	y^5
x^{10}	y^{10}
x^8	y^8
x^2	y^2

Bagging (Bootstrap Aggregating)

- **Bootstrapping**

- 이론적으로 한 개체가 하나의 붓스트랩에 한번도 선택되지 않을 확률

$$p = \left(1 - \frac{1}{N}\right)^N \rightarrow \lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = e^{-1} = 0.368$$

Bagging (Bootstrap Aggregating)

■ Result Aggregating

■ For classification problem

- Majority voting

$$Ensemble(\hat{y}) = \operatorname{argmax}_i \left(\sum_{j=1}^n I(\hat{y}_j = i), i \in \{0,1\} \right)$$

Training Accuracy	Ensemble population	P(y=1) for a test instance	Predicted class label
0.80	Model 1	0.90	1
0.75	Model 2	0.92	1
0.88	Model 3	0.87	1
0.91	Model 4	0.34	0
0.77	Model 5	0.41	0
0.65	Model 6	0.84	1
0.95	Model 7	0.14	0
0.82	Model 8	0.32	0
0.78	Model 9	0.98	1
0.83	Model 10	0.57	1

$$\sum_{j=1}^n I(\hat{y}_j = 0) = 4$$

$$\sum_{j=1}^n I(\hat{y}_j = 1) = 6$$

$$Ensemble(\hat{y}) = 1$$

Bagging (Bootstrap Aggregating)

■ Result Aggregating

■ For classification problem

- Weighted voting (weight = training accuracy of individual models)

$$Ensemble(\hat{y}) = \operatorname{argmax}_i \left(\frac{\sum_{j=1}^n (TrainAcc_j) \cdot I(\hat{y}_j = i)}{\sum_{j=1}^n (TrainAcc_j)}, i \in \{0, 1\} \right)$$

Training Accuracy	Ensemble population	P(y=1) for a test instance	Predicted class label	
0.80	Model 1	0.90	1	
0.75	Model 2	0.92	1	
0.88	Model 3	0.87	1	
0.91	Model 4	0.34	0	
0.77	Model 5	0.41	0	
0.65	Model 6	0.84	1	
0.95	Model 7	0.14	0	
0.82	Model 8	0.32	0	
0.78	Model 9	0.98	1	
0.83	Model 10	0.57	1	
				$\frac{\sum_{j=1}^n (TrainAcc_j) \cdot I(\hat{y}_j = 0)}{\sum_{j=1}^n (TrainAcc_j)} = 0.424$
				$\frac{\sum_{j=1}^n (TrainAcc_j) \cdot I(\hat{y}_j = 1)}{\sum_{j=1}^n (TrainAcc_j)} = 0.576$
				$Ensemble(\hat{y}) = 1$

Bagging (Bootstrap Aggregating)

■ Result Aggregating

■ For classification problem

- Weighted voting (weight = predicted probability for each class)

$$Ensemble(\hat{y}) = \operatorname{argmax}_i \left(\frac{1}{n} \sum_{j=1}^n P(y = i), i \in \{0, 1\} \right)$$

Training Accuracy	Ensemble population	P(y=1) for a test instance	Predicted class label
0.80	Model 1	0.90	1
0.75	Model 2	0.92	1
0.88	Model 3	0.87	1
0.91	Model 4	0.34	0
0.77	Model 5	0.41	0
0.65	Model 6	0.84	1
0.95	Model 7	0.14	0
0.82	Model 8	0.32	0
0.78	Model 9	0.98	1
0.83	Model 10	0.57	1

$$\frac{1}{10} \sum_{j=1}^{10} P(y = 0) = 0.371$$

$$\frac{1}{10} \sum_{j=1}^{10} P(y = 1) = 0.629$$

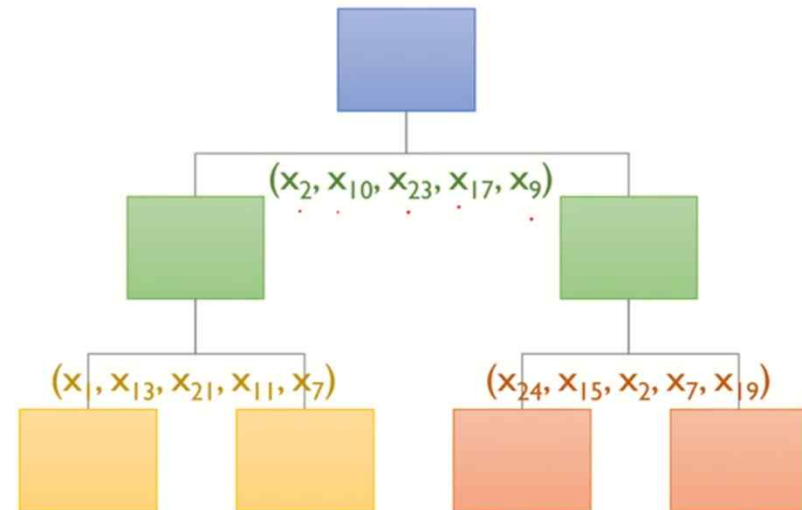
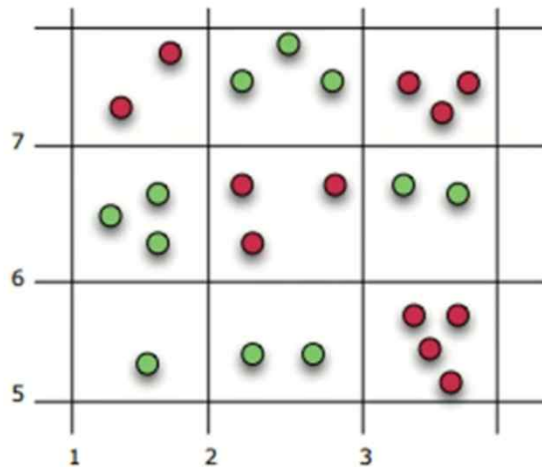
$$Ensemble(\hat{y}) = 1$$

Random Subspace

❖ Random Subspace

- 의사결정나무의 분기점을 탐색할 때, 원래 변수의 수보다 적은 수의 변수를 임의로 선택하여 해당 변수들만을 고려 대상으로 함
- 예시:

Bootstrap i (X in R^{25})



랜덤 포레스트- Generalization Error

- 각각의 개별 tree는 과적합 될 수 있음
- Random forest는 tree수가 충분히 많을 때 Strong Law of Large Numbers에 의해 과적합 되지 않고 그 에러는 limiting value에 수렴됨

$$\text{Generalization error} \leq \frac{\bar{\rho}(1 - s^2)}{s^2} \rightarrow \text{Generalization error의 Upper bound}$$

(작으면 작을 수록 좋아)

$\bar{\rho}$: Decision tree 사이의 평균 상관관계

s : 올바르게 예측한 tree와 잘못 예측한 tree수 차이의 평균

- 개별 tree의 정확도가 높을수록 s 증가
- **Bagging**과 **random subspace** 기법은 각 모델들의 독립성, 일반화, 무작위성을 최대화 시켜 모델간의 상관관계 ρ 를 감소시킴
- 개별 Tree의 정확도, 독립성이 높을수록 random forest의 성능이 높아짐

랜덤 포레스트- 중요변수선택

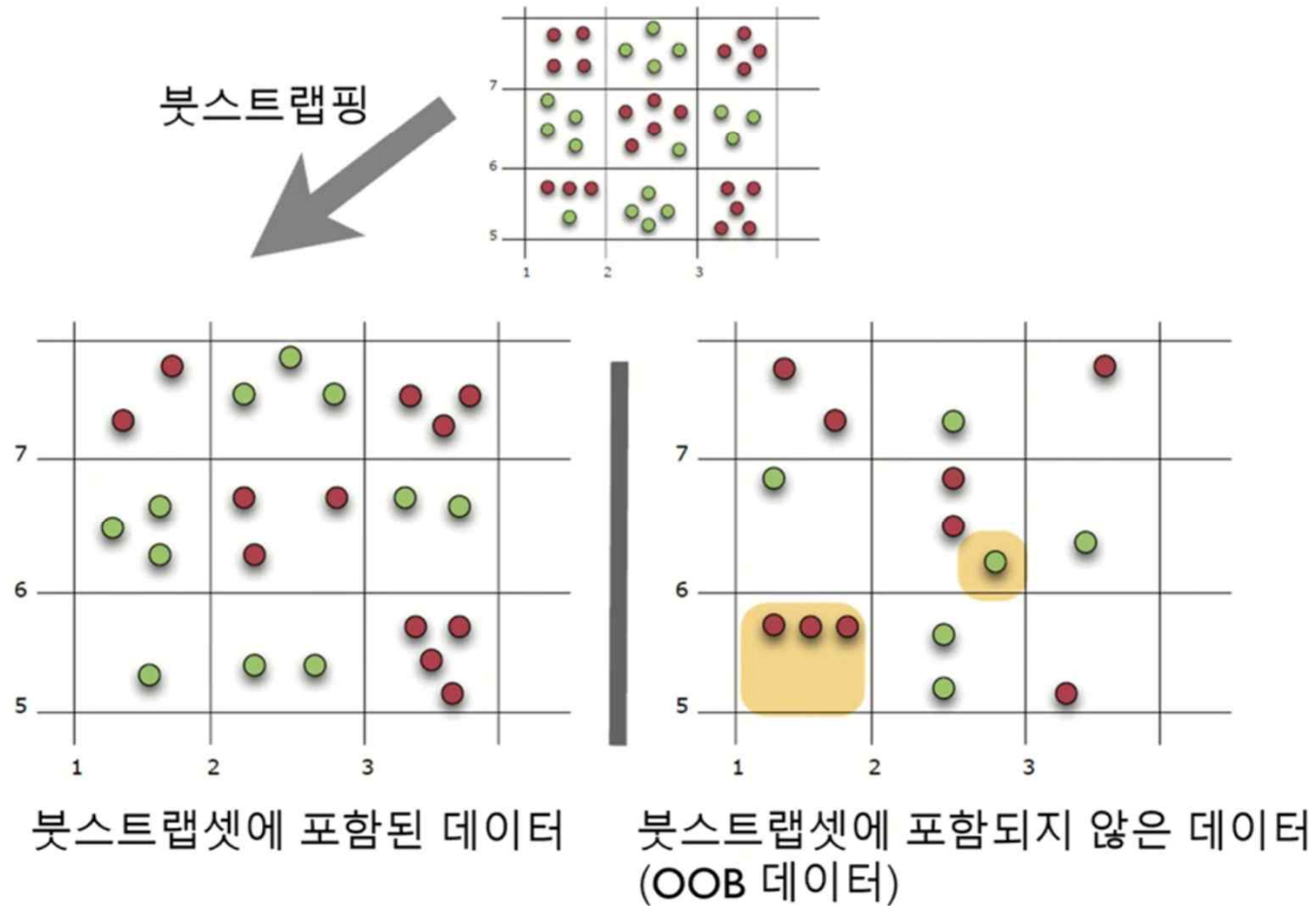
■ 변수의 중요도

- 랜덤 포레스트는 선형 회귀모델/로지스틱 회귀모델과는 달리 개별 변수가 통계적으로 얼마나 유의한지에 대한 정보를 제공하지 않음 ($\because$ 알려진 확률분포를 가정하지 않음)
- 대신 랜덤 포레스트는 다음과 같은 간접적인 방식으로 변수의 중요도를 결정
 - 1단계: 원래 데이터 집합에 대해서 Out of bag (OOB) Error를 구함
 - 2단계: 특정 변수의 값을 임의로 뒤섞은 데이터 집합에 대해서 OOB Error를 구함
 - 3단계: 개별 변수의 중요도는 2단계와 1단계 OOB Error 차이의 평균과 분산을 고려하여 결정

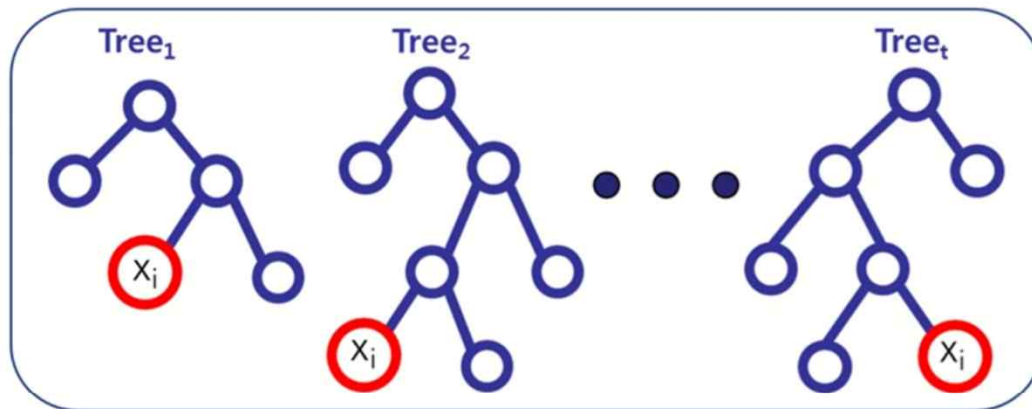
랜덤 포레스트- 중요변수선택

■ Out of bag (OOB)

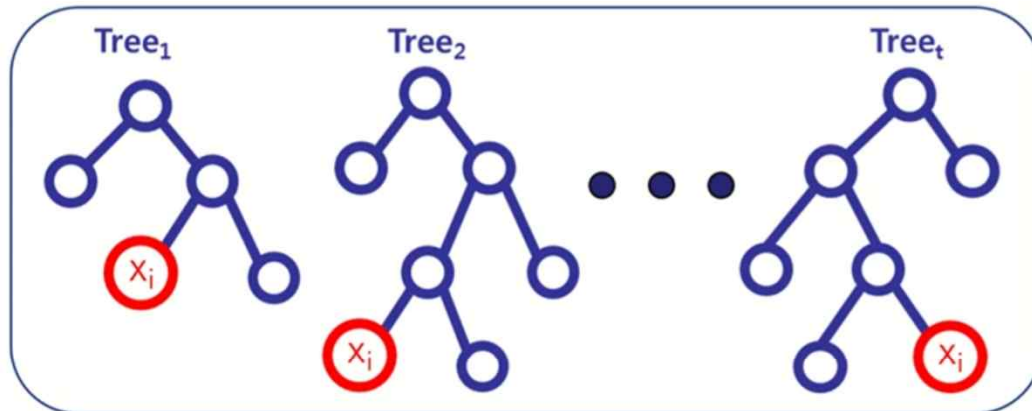
- 배깅을 사용할 경우 붓스트랩셋에 포함되지 않는 데이터들을 검증 집합으로 사용함



랜덤 포레스트- 중요변수선택



(1) 각 붓스트랩셋으로부터 생성된 Tree 에서 OOB error 계산 ($r_i, i=1,2,\dots,t$)



(2) Step (1) Tree에서 특정변수 (X_i)의 값을 임의로 뒤섞은 데이터집합에 대해 OOB error 계산 ($e_i, i=1,2,\dots,t$)

(3) $d_i = e_i - r_i, i = 1, 2, \dots, t$

$$\bar{d} = \frac{1}{t} \sum_{i=1}^t d_i, S_d^2 = \frac{1}{t-1} \sum_{i=1}^t (d_i - \bar{d})^2$$

(4) X_i 변수 중요도: $v_i = \frac{\bar{d}}{S_d}$

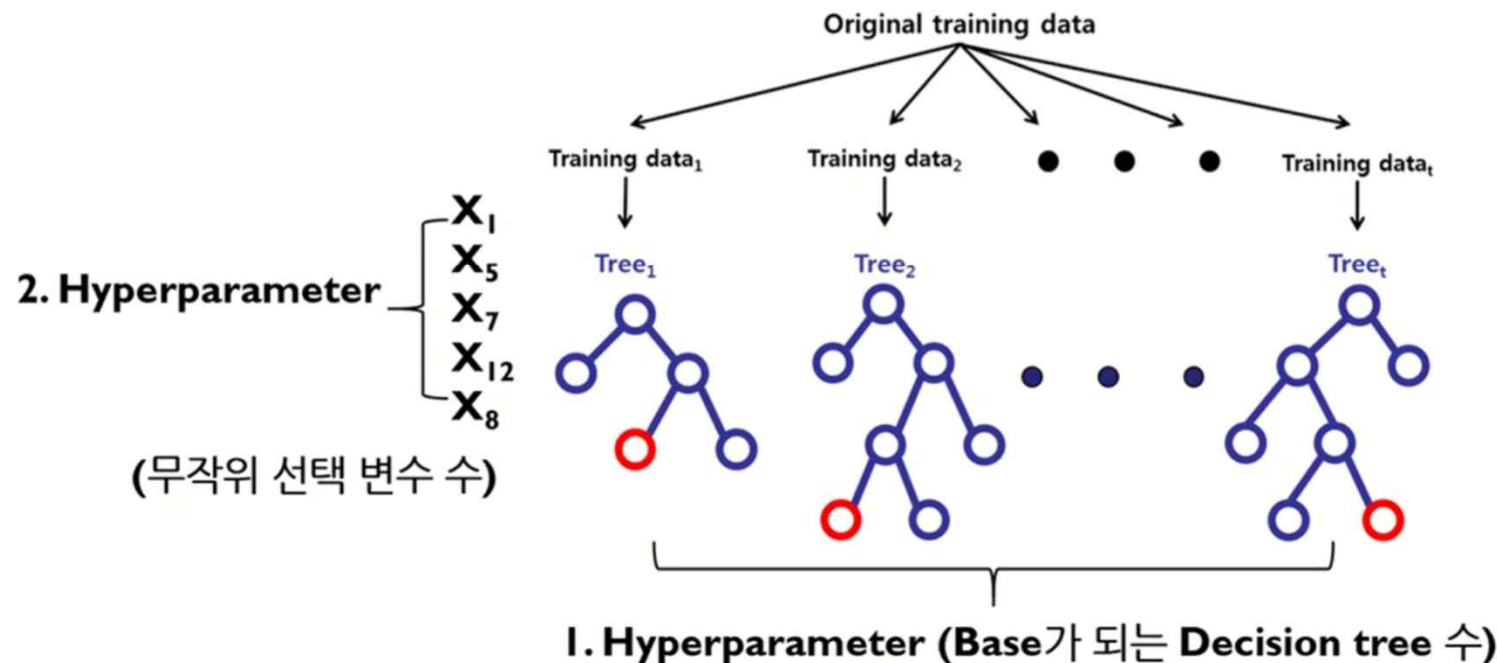
랜덤 포레스트- 하이퍼파라미터

1. Decision tree의 수

- Strong law of large numbers 을 만족 시키기 위해 2,000개 이상의 decision tree 필요

2. Decision tree에서 노드 분할 시 무작위로 선택되는 변수의 수

- 일반적으로 변수의 수에 따라 다음과 같이 추천됨 (Diaz-Uriarte et al., 2006)
- Classification: $\sqrt{\text{변수의 수}}$
- Regression: 변수의 수/3



실습

< 비교 >

Max_features= ' sqrt ' 와
max_features=None

'sqrt' : Square Root

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.datasets import make_classification
6 from sklearn.model_selection import train_test_split
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.metrics import accuracy_score, classification_report
9
10 # 1. 스마트 팩토리 센서 데이터 시뮬레이션
11 # 총 36개의 센서(변수) 중 5개만 실제 불량과 직결된 '핵심 센서'로 설정
12 X, y = make_classification(n_samples=1500, n_features=36, n_informative=5,
13                             n_redundant=2, random_state=42)
14 # 훈련 데이터와 테스트 데이터 분리 (8:2)
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
16
17 # 2. 랜덤 포레스트 모델 구축 및 하이퍼파라미터 세팅 (★ 핵심)
18 rf_model = RandomForestClassifier(
19     n_estimators=500,          # [대수의 법칙] 500그루의 나무 생성 (연산 자원에 따라 2000까지 세팅)
20     max_features='sqrt',      # [랜덤 서브스페이스] 분류 문제의 국룰인 루트 p 적용 (36개 중 6개 무작위 선택)
21     oob_score=True,           # [검증] OOB(Out-of-Bag) 모의고사 평가 활성화
22     random_state=42,
23     n_jobs=-1                  # 병렬 처리 (6CPU 및 멀티코어 리소스 100% 활용)
24 )
25
26 # 3. 모델 학습
27 rf_model.fit(X_train, y_train)
28
29 # 4. 검증 및 평가
30 # OOB Score 출력 (버려진 36.8% 데이터로 자체 모의고사를 본 결과)
31 print(f"OOB Score (자체 검증 점수): {rf_model.oob_score_:.4f}")
32
33 # 테스트 데이터 추론 및 최종 평가
34 y_pred = rf_model.predict(X_test)
35 print(f"Test Accuracy (실전 테스트 점수): {accuracy_score(y_test, y_pred):.4f}")
36 print("\n[분류 리포트]")
37 print(classification_report(y_test, y_pred))
38
39 # 5. Feature Importance (변수 중요도) 시각화
40 # 노이즈 제거(Permutation) 기반으로 산출된 각 센서의 중요도 점수
41 importances = rf_model.feature_importances_
42 indices = np.argsort(importances)[::-1] # 내림차순 정렬
43
44 # 상위 10개의 핵심 센서만 시각화
45 plt.figure(figsize=(10, 6))
46 plt.title("Smart Factory Sensor Feature Importances (Top 10)")
47 plt.bar(range(10), importances[indices[:10]], align="center", color='steelblue')
48 plt.xticks(range(10), [f"Sensor_{i}" for i in indices[:10]], rotation=45)
49 plt.xlim([-1, 10])
50 plt.ylabel("Importance Score")
51 plt.tight_layout()
52 plt.show()
```